

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт

з лабораторної роботи №4

з дисципліни: «Поглиблене вивчення JavaScript»

з теми: «Розробка застосунку реєстрації на події (розширені можливості Vue)»

Виконав:

ст. гр. ПЗП-23-3

Білоус А. А.

Перевірив:

доц. каф. ПІ

Мар'їн С. О.

4 РОЗРОБКА ЗАСТОСУНКУ РЕЄСТРАЦІЇ НА ПОДІЇ (РОЗШИРЕНІ МОЖЛИВОСТІ VUE)

4.1 Мета роботи

Закріплення та поглиблення знань з розробки застосунків на Vue 3, засвоєння принципів використання Composition API, організації логіки за допомогою composables, реалізація багатосторінкових застосунків, а також формування навичок роботи з асинхронними операціями та збереженням даних.

4.2 Хід роботи

4.2.1 Структура проєкту

Застосунок побудовано на Vite + Vue 3 з Vue Router. Файлова структура:

- «src/router/index.js» – конфігурація Vue Router;
- «src/composables/useLocalStorage.js» – composable для реактивного localStorage;
- «src/composables/useEvents.js» – composable зі списком подій;
- «src/composables/useRegistration.js» – composable з логікою реєстрації;
- «src/components/EventCard.vue» – компонент банера події;
- «src/views/EventListView.vue» – сторінка списку подій;
- «src/views/EventDetailView.vue» – сторінка деталей події;
- «src/views/RegisterView.vue» – сторінка реєстрації;
- «src/App.vue» – кореневий компонент із навігацією та «<RouterView>».

4.2.2 Маршрутизація

Vue Router налаштовано з чотирма маршрутами. «EventListView» обгорнуто у «<KeepAlive>» в «App.vue», що зберігає стан компонента при переходах між сторінками:

```
const routes = [
  { path: '/', redirect: '/events' },
  { path: '/events', component: EventListView },
  { path: '/events/:id', component: EventDetailView },
  { path: '/events/:id/register', component: RegisterView },
]
```

```

<RouterView v-slot="{ Component }">
  <KeepAlive :include="['EventListView']">
    <component :is="Component" />
  </KeepAlive>
</RouterView>

```

4.2.3 Composable «useLocalStorage»

Базовий composable, що повертає реактивний «ref», синхронізований із «localStorage» через «watch»:

```

export function useLocalStorage(key, defaultValue) {
  const stored = localStorage.getItem(key)
  const value = ref(stored !== null ? JSON.parse(stored) :
defaultValue)

  watch(value, (val) => {
    localStorage.setItem(key, JSON.stringify(val))
  }, { deep: true })

  return value
}

```

4.2.4 Composable «useEvents»

Зберігає масив подій (кожна з полями «id», «title», «type», «date», «tagline», «description», «gradient») та надає функцію пошуку:

```

export function useEvents() {
  const events = ref(EVENTS)
  function findEvent(id) {
    return events.value.find(e => e.id === Number(id)) ?? null
  }
  return { events, findEvent }
}

```

4.2.5 Composable «useRegistration»

Ключовий composable з повним набором функціональності. На рівні модуля (singleton) зберігаються два об'єкти: «allRegistrations» через «useLocalStorage» та спільний «toast», доступний усім компонентам, що викликають composable:

```

const allRegistrations = useLocalStorage(STORAGE_KEY, {})
const toast = ref(null)

```

Асинхронна функція «register» реалізує оптимістичне оновлення: запис додається до списку негайно, ще до відповіді сервера. У разі помилки виконується відкат:

```
async function register(name, email) {
  const prev = [...(allRegistrations.value[eventId] ?? [])]
  const entry = { name, email, registeredAt: new
Date().toISOString() }

  allRegistrations.value = { ...allRegistrations.value, [eventId]:
[...prev, entry] }
  isLoading.value = true

  try {
    await fakeApiRegister()
    showToast('success', `${name}, вас успішно зареєстровано!`)
  } catch (err) {
    allRegistrations.value = { ...allRegistrations.value, [eventId]:
prev }
    showToast('error', err.message)
  } finally {
    isLoading.value = false
  }
}
```

4.2.6 Сторінка списку подій

«EventListView.vue» отримує масив подій через «useEvents()» і відображає їх у CSS Grid (3 колонки). Для KeepAlive компонент іменовано через «defineOptions»:

```
defineOptions({ name: 'EventListView' })
const { events } = useEvents()
```

4.2.7 Компонент EventCard

«EventCard.vue» відображає кольоровий банер події з градієнтним фоном, назвою та tagline. Пропс «size» визначає висоту («sm» – 130px, «lg» – 240px), «rounded» – чи мати заокруглення з усіх боків.

4.2.8 Сторінка деталей події

«EventDetailView.vue» знаходить подію за «route.params.id», відображає інформацію у двоколонковому макеті та список зареєстрованих учасників через «registrations» з «useRegistration»:

```
const eventId = Number(route.params.id)
const event = computed(() => findEvent(eventId))
const { registrations, toast } = useRegistration(eventId)
```

4.2.9 Сторінка реєстрації

«RegisterView.vue» містить форму з двома полями («name», «email») та валідацією: перевірка обов’язкових полів, формату email і дублювання. Реалізовано справжній оптимістичний інтерфейс: після валідації «register()» запускається без «await», а навігація на сторінку деталей відбувається негайно. Користувач бачить себе у списку одразу, не очікуючи відповіді сервера:

```
function submit() {
  if (!validate()) return
  register(form.value.name.trim(), form.value.email.trim())
  router.push(`/events/${eventId}`)
}
```

4.2.10 Teleport для сповіщень

Оскільки тост відображається вже на «EventDetailView» (куди користувач переходить до завершення запиту), «toast» винесено на рівень модуля у «useRegistration» – це забезпечує його доступність після навігації. Сам елемент рендериться безпосередньо в «<body>» через «<Teleport>», що унеможливлює проблеми з «z-index»:

```
<Teleport to="body">
  <div v-if="toast" class="toast-fixed" :class="toast.type">
    {{ toast.message }}
  </div>
</Teleport>
```

4.3 Висновки

У ході лабораторної роботи розроблено повнофункціональний застосунок реєстрації на події. Три composable інкапсулюють окремі зони відповідальності: «useLocalStorage» – реактивне збереження, «useEvents» – дані подій, «useRegistration» – асинхронна логіка реєстрації з оптимістичним оновленням та відкатом. Vue Router забезпечує навігацію між трьома сторінками, «<KeepAlive>»

кешує список подій, «<Teleport>» виносить тост-сповіщення до «<body>». Дані про реєстрації зберігаються між сесіями через «localStorage».

4.4 Відповіді на контрольні запитання

- а) **Що таке Composition API?** Підхід до організації логіки у Vue 3, при якому пов'язаний код групується разом у функціях «setup()». На відміну від Options API (де логіка розподіляється по «data», «methods», «computed»), Composition API дозволяє виокремлювати функціонал у composables та повторно використовувати його в різних компонентах.
- б) **Що таке composables?** Функції, що інкапсулюють і повертають реактивну логіку із використанням Composition API. Composable «use*» може приймати параметри, тримати внутрішній стан, виконувати side-effects і повертати реактивні значення та методи. Це основний механізм повторного використання логіки у Vue 3.
- в) **Призначення Vue Router.** Бібліотека клієнтської маршрутизації для Vue-застосунків. Пов'язує URL-адреси з компонентами-сторінками (views), забезпечує навігацію без перезавантаження сторінки, надає реактивний доступ до параметрів маршруту через «useRoute()» та програмну навігацію через «useRouter()».
- г) **Як реалізується робота з localStorage?** Через «useLocalStorage» composable: при ініціалізації значення зчитується з «localStorage» та поміщається у «ref». «watch» із «{ deep: true }» відстежує будь-які зміни та серіалізує оновлене значення назад у сховище. Завдяки цьому будь-який компонент, що використовує цей composable, автоматично отримує синхронізований із localStorage реактивний стан.
- д) **Що таке оптимістичний інтерфейс?** Підхід UX, при якому зміна відображається в інтерфейсі негайно – до того, як сервер підтвердить операцію. У застосунку це реалізовано так: після валідації форми «register()» запускається без очікування, а маршрутизатор одразу переводить користувача на сторінку деталей, де новий учасник вже видний у списку. Якщо сервер повертає помилку, запис зникає зі списку (rollback) і з'являється тост із поясненням.

- е) **Як обробляються помилки в асинхронних операціях?** Через «try/catch/finally» у «async» функціях. У блоці «try» виконується оптимістичне оновлення та виклик «await fakeApiRegister()». У «catch» перехоплюється помилка: відновлюється попередній стан (rollback) і відображається тост із повідомленням. Блок «finally» завжди скидає «isLoading» у «false» незалежно від результату.